Counter Artificial Intelligence - A Preliminary Analysis of the Capabilities of AI Models on Local Systems

J. Wolffrom

jwsc@ruc.dk

Supervisor:

Hemant Ghayvat

hghayvat@ruc.dk

May 28, 2026

Roskilde University

Department of Computer Science

Abstract

Contents

1 Glossary

list of terms and definitions:

- **Process** - An instance of a program that is being executed. It contains the program code and its current activity.
- **Pipeline** - A series of data processing stages where the output of one stage is the input to the next.
- **Thread** - An isolated unit of execution that shares memory space with other threads of the same process.
- **Job** - A process or a pipeline that is started or scheduled to run by the user in a shell session or from the terminal.

2 Disclosure

Any and all resources related to- and used in this study can be found here:

https://github.com/X1las/CAI

Everything in this paper has been written by human hands and proofread by Claude 4.5 Sonnet.

AI has been used for spell checking, sentence structure corrections, as well as text reductions, however everything has been checked by a human to make sure edits do not conflict with the cited theory.

The list of commands in appendix A was sourced from SS64 n.d., and compiled/categorized by the previously mentioned model.

3 Introduction

In August 2025, Anthropic's Claude model was reportedly used in one of the first documented cases of "vibe hacking" as outlined in Alex Moix, J. K., Ken Lebedev. (2025). *Threat intelligence report: August 2025*. Anthropic. In that case, the attacker used the model to collect publicly available vulnerability information, identify organizations using the affected software, and exploit the vulnerability to gain access to internal infrastructure. The incident was notable because it required limited human involvement, while the model assisted in producing malicious tooling used for extortion.

If AI continues to develop at the current pace, similar incidents are likely to become more frequent. It is also plausible that some future attacks will be increasingly autonomous, if not fully. In that context, the need for stronger cybersecurity capabilities will grow, and many experts argue that AI will play a central role in both attack and defense strategies. This motivates research into how AI models behave when they share the same system and are tasked with detecting, disrupting, or countering each other.

3.1 The Problem

The aim of this research paper is not to predict exactly how AI models would counteract each other, but to explore how they might use available system tools in a simulated local environment, and whether useful insights can be gained from this process.

In simple terms, the problem is twofold. First, it is unclear which terminal commands are realistically available to an AI agent running locally with broad system access. Second, it is not known how an AI model should be trained to use such commands effectively, or which combinations are most useful for detecting and counteracting another model in the same environment.

3.1.1 Research Question

The research question is then as follows:

How can AI agents detect and counteract each other through Linux filesystem operations and process management commands in a local system environment?

- **1.** What Linux filesystem and process management commands (including `ps`, `kill`, `fuser`, file operations, and permission management) can be used by an AI agent to monitor, identify, and interact with another AI agent on the same system?
- **2.** How can filesystem-based communication mechanisms (including file creation, modification, symbolic links, and access control) serve as both attack vectors and detection methods in AI-to-AI interactions?
- **3.** What strategies emerge when two AI agents are simulated in a Linux environment with access to process monitoring, job scheduling, and file manipulation commands, and what patterns of behavior can be identified?
- **4.** What are the implications of filesystem-centric AI defense mechanisms for the future of autonomous cybersecurity systems?

To address these questions, we will first compile a list of terminal commands on linux that can be used to outline a base of actions for the simulation. This will be limited to linux for the sake of simplicity, but the process can be expanded to include Windows commands as well.

3.2 Scope and Limitations

This research is intentionally scoped to focus on the Linux operating system and its associated filesystem operations. All commands, system calls, utilities, and code examples presented in this paper relate specifically to the Linux environment and its filesystem-centric architecture.

This limitation has several implications. First, the commands and techniques discussed are primarily applicable to Linux-based systems and may not directly translate to other operating systems such as Windows or macOS, which have fundamentally different filesystem structures, permission models, and command-line interfaces. Second, while Linux systems support various network operations and inter-process communication mechanisms, this study focuses on filesystem-based interactions, as these represent the most fundamental and universally available means of system manipulation and inter-agent communication.

The rationale for this scope is threefold: (1) Linux provides a well-documented, standardized command-line environment that is widely used in server and embedded systems where autonomous AI agents are most likely to operate; (2) the filesystem serves as a common substrate through which processes can interact, persist data, and observe system state, making it a natural focus for studying AI-to-AI interactions; and (3) limiting the scope to a single operating system allows for deeper analysis of specific commands and techniques rather than superficial coverage of multiple platforms.

Consequently, this paper does not address platform-specific considerations for Windows (e.g., PowerShell, Registry manipulation, NTFS permissions) or macOS (e.g., launchd, Keychain access, APFS features). Additionally, while network-based attacks and defenses are acknowledged as important vectors in real-world scenarios, they fall outside the scope of this filesystem-focused investigation.

4 Linux Commandline

A list of the linux commands used in this paper is included in Appendix A. This list is not an exhaustive list of all commands available on linux based distributions, it is based on the commands found on SS64 (n.d.), which includes most if not all the built-in bash commands. For a full list of linux commands, see Kerrisk (n.d.).

To begin with, we can derive that commands on the linux commandline can affect most of anything on the system. Processes, files, network connections, users and even hardware can be affected by commands. This means that in order to understand what can be done locally, we need to understand how the system works at its most fundamental level and what commands gives way to manipulate.

4.1 Processes and Jobs

Processes are a fundamental unit of execution in the Linux operating system, the management of which oversees all actions taken on the system and by the system. Understanding how these processes work, how they can be monitored and affected by the user, commands, and other processes is crucial in understanding how AI agents could operate in our simulated environment, and how they could potentially interact with each other.

For references on processes we refer to our previous work *Asynchronous Synergy: A study of performance in Multi-Core Threading across programming languages* (Wolffrom 2025), this report will not cover the finer details of process or thread management, but rather what they mean for the commandline.

4.1.1 Process Architecture and Concepts

There are four important keywords in this category, which are *processes*, *threads*, *pipelines* and *jobs*. Processes can be somewhat loosely defined as we discovered in our prior study (Wolffrom 2025), however they are generally considered to be instances of code that are running in an isolated memory space. Each process has its own virtual address space, preventing it from directly accessing the memory of other processes and this isolation is enforced by the kernel and provides fundamental security boundaries between processes.

When a process is created through the `fork()` system call, it receives a unique Process ID (PID). The newly created child process is initially an exact copy of the parent process, including its memory space, open file descriptors, and execution context. The child can then use `exec()` to replace its memory space with a new program. This fork-exec model is fundamental to how shells launch commands and how processes spawn subprocesses.

Threads can be generally split into two categories, virtual threads (also known as user-level threads) and kernel threads, where the former are managed by the process itself and the latter are managed by the operating system's scheduler, which is usually EEVDF (Community 2025, EEVDF Scheduler) in the linux environment. Processes are usually scheduled to run on kernel threads, and the kernel threads are scheduled to have equal time allotted on the CPU. Multiple threads within a single process share the same memory space, making inter-thread communication faster but also introducing synchronization challenges.

Processes maintain several important attributes beyond their PID, including a Parent

Process ID (PPID) that identifies which process created them, a user ID (UID) and group ID (GID) that determine privileges, and various resource limits. When a parent process terminates before its children, those child processes are typically reparented to the init process (PID 1), becoming orphaned processes. Conversely, when a child terminates but the parent has not yet read its exit status, it becomes a zombie process, retaining an entry in the process table until the parent collects its status.

4.1.2 Pipelines and Process Communication

A pipeline in the context of the linux commandline refers to the mechanism of connecting the output of one process to the input of another process using the `|` operator ("Piping in Unix or Linux" 2024). This is not related to the concept of pipelines in the context of data processing or machine learning, but it is a powerful tool for chaining commands together and creating complex workflows. An example of this would be the command `ps aux | grep python`, which would list all the processes running on the system and then filter the results to show only those that contain the word "python". This allows for a user to easily search through the processes and find specific ones that they are interested in.

Pipelines are implemented using anonymous pipes, which are unidirectional data channels managed by the kernel. When a shell creates a pipeline, it forks multiple child processes and connects their standard output and input streams. The processes in a pipeline execute concurrently, with data flowing between them as it becomes available. All processes in a pipeline share the same process group ID (PGID), which allows them to be managed collectively.

Beyond anonymous pipes, Linux provides several other inter-process communication (IPC) mechanisms. Named pipes (FIFOs) persist in the filesystem and can be used by unrelated processes. Shared memory allows multiple processes to access the same memory regions for high-performance data exchange. Message queues provide structured communication channels. Sockets enable communication both locally and across networks. Understanding these mechanisms is crucial because they represent potential channels through which AI agents might exchange information or detect each other's presence.

4.1.3 Job Control and Management

By cross referencing "Cron Command in Linux with Examples" (2026), Free Software Foundation (2025, May), "Job Scheduling Commands in Linux" (2025), and "Shell Scripting - JOB SPEC & Command" (2025), jobs are a concept in the linux commandline manuals that refer to processes and pipelines that are either started or scheduled to run by the user in a shell session or from the terminal. They are given a process ID (PID) on creation, regardless of whether they are multiple commands tied together in a pipeline or a single command. They can be run in the foreground or background, and they can be managed using built-in commands such as `jobs`, `fg`, `bg`, and `kill`.

Foreground jobs receive keyboard input and prevent the shell from accepting new commands until they complete or are suspended. Background jobs, launched by appending `&` to a command, run concurrently with the shell, allowing the user to continue issuing commands. The `Ctrl-Z` key combination suspends a foreground job, which can then be resumed in the background with `bg` or returned to the foreground with `fg`.

Job control is mediated through signals, which are asynchronous notifications sent to processes. Common signals include SIGTERM (polite termination request), SIGKILL (immediate forced termination), SIGSTOP (suspend execution), and SIGCONT (resume execution). The `kill` command, despite its name, is the primary interface for sending arbitrary signals to processes.

Using cron, a user can also schedule jobs to run at specific times or intervals, which can be useful for automating tasks and running processes without user intervention. The `crontab` command manages per-user cron schedules, while system-wide cron jobs are defined in `/etc/crontab` and `/etc/cron.d/`. The `at` and `batch` commands provide one-time scheduled execution. These scheduling mechanisms could allow an AI agent to persist its presence or execute actions at predetermined times.

Where processes usually differ from jobs is that they can be triggered and scheduled by the operating system without user involvement. Jobs are generally more manageable for users, as they are directly tied to the shell and can be easily manipulated using built-in commands. Processes, on the other hand, can be more complex and may require more advanced tools for management and monitoring. Due to their nature as well, processes can be more anonymous and harder to track as well, seeing as they can be triggered by other processes and may not have a direct association with a user or shell session.

4.1.4 Process Monitoring and Discovery

Commands such as `ps`, `top`, `htop` and `pgrep` can be used to monitor and search through processes, whilst commands such as `pkill`, `killall` in addition to `kill` all allow for termination of processes.

The `ps` command provides a snapshot of current processes. With options like `aux`, it displays all processes system-wide with detailed information including CPU and memory usage, execution time, and the command line used to launch each process. The `-ef` option provides a full-format listing showing parent-child relationships through PPID values. For AI agents, examining process command lines and parent relationships could reveal the presence of competing agents or suspicious activity.

The `top` and `htop` commands provide real-time, continuously updating views of system processes, sorted by various metrics such as CPU or memory consumption. These interactive tools highlight processes that are consuming unusual amounts of resources, which could indicate malicious behavior or intensive computational tasks associated with an AI agent.

The `pgrep` command searches for processes by name or other attributes, returning only the PIDs. Its counterpart `pkill` sends signals to matching processes. The `pidof` command finds PIDs by exact command name. The `pstree` command displays the process hierarchy as a tree, making parent-child relationships visually apparent.

More sophisticated process introspection is available through the `/proc` pseudo-filesystem, where each process has a directory named by its PID containing files with detailed information: `/proc/[pid]/cmdline` shows the command line, `/proc/[pid]/exe` is a symlink to the executable, `/proc/[pid]/environ` contains environment variables, and `/proc/[pid]/fd/` lists open file descriptors. An AI agent with filesystem access could extensively analyze other processes through this interface.

4.1.5 Process Control and Termination

Beyond the basic `kill` command, Linux provides several mechanisms for process termination. The `killall` command sends signals to all processes matching a given name, while `pkill` offers more flexible pattern matching and attribute-based selection. The `timeout` command can be used to automatically terminate a process if it exceeds a specified execution time.

Process resource limits can be queried and modified using `ulimit` (for the current shell and its children) or `prlimit` (for arbitrary processes). These limits cover maximum file sizes, number of open files, memory usage, CPU time, and other resources. An AI agent could potentially use these mechanisms to constrain a competing agent's capabilities or to detect constraints imposed upon itself.

The `strace` command traces system calls made by a process, revealing every interaction with the kernel including file operations, network activity, and process management. Similarly, `ltrace` traces library calls. These powerful debugging tools could allow one AI agent to monitor another's behavior in detail, though they require appropriate privileges.

4.1.6 Process Scheduling and Priority Management

In addition to this, the EEVDF scheduler (Community 2025, EEVDF) allows for scheduling affinity using the `sched_setattr()` system call, which the user can call on directly on starting a process using the `nice` command, or after the process has started with `renice`. This allows for a user to give a process higher or lower priority on the CPU, which can make some processes run with more CPU time than others and vice versa. This can be used to give a process more resources to run, or to starve a process of resources and make it run slower.

Process priorities in Linux are expressed as "nice" values ranging from -20 (highest priority) to 19 (lowest priority), with 0 being the default. Despite the name, higher nice values are "nicer" to other processes by yielding more CPU time to them. Only privileged users can decrease nice values (increase priority), though any user can increase nice values (decrease priority) for their own processes.

Beyond nice values, Linux supports multiple scheduling classes. The default SCHED_OTHER (or SCHED_NORMAL) uses the EEVDF algorithm for time-sharing. Real-time scheduling classes (SCHED_FIFO and SCHED_RR) provide deterministic scheduling for time-critical applications but require elevated privileges. The `chrt` command can query and modify scheduling policies and priorities.

CPU affinity, controlled by `taskset`, restricts a process to execute only on specified CPU cores. This can be used for performance optimization but could also be weaponized to isolate or constrain competing processes. The `cgroups` (control groups) mechanism provides comprehensive resource management, allowing fine-grained control over CPU time, memory allocation, I/O bandwidth, and other resources for groups of processes.

4.1.7 Process Dependencies and Synchronization

In addition to these, there are also commands such as `wait` which can be used to pause the execution of a process until another process has finished, and `fuser` which can be used to identify which processes are using a specific file or socket. This can be useful for identifying which processes are interacting with each other and potentially interfering with each other.

The `wait` command is primarily used in shell scripts to synchronize parent processes with their children. Without `wait`, a parent might terminate before its children complete, or might not know whether child processes succeeded or failed. The command can wait for specific PIDs or for all background jobs.

The `fuser` command identifies processes using specific files or sockets, returning PIDs and access types (read, write, execute, or root directory). This is particularly valuable for detecting filesystem-based interactions between processes. The related `lsof` (list open files) command provides comprehensive information about all files opened by processes, including regular files, directories, network sockets, pipes, and devices. Since everything in Unix-like systems is treated as a file, `lsof` offers a powerful view into process behavior and inter-process relationships.

File locking mechanisms, accessible through the `flock` command or programmatically through system calls, allow processes to coordinate access to shared resources. An AI agent could use file locks both for legitimate synchronization and for detecting or blocking competing agents attempting to access the same resources.

4.2 User Management and Groups

5 The Linux Filesystem

Filesystems are the organizational structures that determine how data is stored and retrieved on a storage device. They define the way files are named, stored, and accessed, as well as how metadata is managed. Common filesystems in Linux include ext4, XFS, Btrfs, and F2FS, each with its own features and performance characteristics.

The Linux filesystem provides the fundamental infrastructure through which processes interact with persistent data and communicate with each other. Understanding file operations is crucial for analyzing how AI agents might manipulate system state, hide information, or detect the presence of other agents.

5.1 Data Blocks, Clusters and Sectors

Sectors refer to the smallest physical unit of storage on a traditional disk drive, typically 512 bytes of storage. Sectors used to be fixed in size due to the limitations and design of hard disk drives, but with modern solid-state drives the notion of sectors has become a logical abstraction rather than a physical constraint.

When a collection of sectors is grouped together, it forms a block/cluster depending on the filesystem. A block is a logical unit of storage that the filesystem uses to manage files. The size of blocks can vary depending on the filesystem and the underlying storage device, but common block sizes are 4KB or 8KB. Clusters are essentially the same as blocks, but they are often used in the context of NTFS filesystems that the Windows operating system uses.

Blocks are the smallest logical unit of storage, meaning that even if a file is only a few bytes in size, it will still occupy at least one block on the filesystem. This can lead to inefficiencies, especially for small files, as the unused space within the block cannot be utilized by other files.

5.1.1 Super Blocks

Blocks are tracked by the filesystem's metadata structures, such as inodes or the superblock, which record which blocks are allocated to which files. When a file is created or modified, the filesystem allocates blocks to store the file's data and updates the metadata accordingly. The block size can impact performance and storage efficiency; larger blocks can reduce fragmentation but may waste space for small files, while smaller blocks can be more efficient for small files but may lead to increased fragmentation.

5.1.2 Boot Blocks

The boot block (or boot sector) is a reserved area at the beginning of a storage device or partition that contains the code necessary to start the boot process. On traditional BIOS systems with MBR (Master Boot Record) partitioning, the boot block occupies the first 512 bytes of the disk and contains both the bootloader code and the partition table. The bootloader is a small program that the system firmware executes to locate and load the operating system kernel into memory.

On modern UEFI (Unified Extensible Firmware Interface) systems with GPT (GUID Partition Table), the boot process is more complex and uses an EFI System Partition

(ESP), which is a dedicated FAT32 partition containing bootloader files. However, GPT disks still maintain a protective MBR in the first sector for backward compatibility with legacy systems.

Boot blocks are critical for system initialization and are protected areas that typically cannot be modified by normal filesystem operations. In multi-boot scenarios, boot managers like GRUB (Grand Unified Bootloader) or systemd-boot reside in these boot blocks and provide menus for selecting which operating system to load. Understanding boot blocks is important in security contexts, as malicious actors might attempt to modify boot code to achieve persistence or execute code before the operating system loads (bootkits or rootkits).

5.2 Inodes

At the filesystem implementation level, files are represented by inodes (index nodes), which are data structures used by many Unix-like filesystems to represent files and directories. Each inode contains metadata about a file, including its size, permissions, ownership, timestamps (creation, modification, access), and pointers to the actual data blocks where the file's content is stored. The inode does not contain the filename itself; instead, directory entries map filenames to their corresponding inode numbers.

When a file is accessed, the filesystem uses the directory structure to resolve the filename to an inode number, then retrieves the inode to access the file's metadata and data blocks. This separation of filename and file metadata allows for features like hard links, where multiple directory entries can point to the same inode, effectively giving a single file multiple names.

5.3 Filesystems and The Virtual File System

Linux organizes storage into three conceptual levels: the virtual file system (VFS)(Richard Gooch n.d.), individual filesystems, and files themselves. The Virtual File System (or VFS for short) is an abstraction layer within the Linux kernel that provides a uniform interface for different filesystem implementations. It allows the kernel to support multiple filesystem types without requiring changes to user-space applications. The VFS provides a unified interface that abstracts the underlying filesystem implementations, allowing different filesystem types to coexist and be accessed through a common set of system calls. The VFS manages file operations, directory structures, and metadata in a way that is consistent across all supported filesystems.

At the top level, Linux employs a single hierarchical directory tree("Unix File System" 2026) rooted at `/`, unlike Windows which uses drive letters. All storage devices, regardless of their physical location or filesystem type, are mounted into this unified tree structure. Common mount points include `/home` for user directories, `/tmp` for temporary files, and `/proc` or `/sys` for kernel-provided pseudo-filesystems that expose process and system information.

5.3.1 Filesystem Types

Individual filesystems can be of various types, each with distinct characteristics. Modern Linux distributions commonly use ext4, which provides journaling for crash recovery and supports large files and volumes. The ext (extended) filesystem family, including ext2, ext3, and ext4, is one of the most widely used filesystems in Linux. Ext4 is the latest version and offers features such as journaling for improved reliability, support for large files and volumes, and enhanced performance. It uses a block-based allocation scheme and supports extents, which allow for more efficient storage of large files.

Alternative filesystems include XFS (optimized for parallel I/O operations), Btrfs (supporting snapshots and copy-on-write), and F2FS (optimized for flash storage). The choice of filesystem affects performance characteristics, maximum file sizes, and available features, though the VFS layer ensures that basic operations remain consistent across types.

5.4 Partitions

Partitions are divisions of a storage device that allow it to be managed separately. Each partition can contain a different filesystem, enabling multiple operating systems or different types of data to coexist on the same physical device. Partitions are defined in a partition table, which is typically located at the beginning of the storage device. Common partitioning schemes include MBR (Master Boot Record) and GPT (GUID Partition Table).

5.5 File Operations

Files can be created, modified, copied, moved, and deleted using a variety of commands. The `touch` command creates empty files or updates timestamps, while `mkdir` creates directories. File content can be written using redirection operators (`>` and `>>`) or commands like `echo`, `cat`, or text editors.

Copying and moving files is accomplished through `cp` and `mv` respectively. The `cp` command creates a new file with duplicated content, while `mv` relocates the file within the filesystem hierarchy without duplicating data when the source and destination are on the same filesystem. Deletion is performed with `rm` for files and `rmdir` or `rm -r` for directories.

More sophisticated operations include `rsync`, which can synchronize files between directories or systems while minimizing data transfer through checksums and delta encoding, and `dd`, which performs low-level block-by-block copying useful for creating disk images or manipulating raw devices.

5.6 Links

This separation of filename and inode enables the concept of links. A hard link is an additional directory entry pointing to the same inode, effectively giving a single file multiple names. All hard links are equivalent, and the file's data is only deleted when the last link is removed and no processes have the file open. Hard links cannot span filesystems and cannot reference directories (to prevent circular references).

Symbolic links (symlinks), created with `ln -s`, are special files containing a path to another file. Unlike hard links, symlinks can reference directories and cross filesystem boundaries, but they break if the target is moved or deleted. The `readlink` command can resolve symbolic links to their targets.

5.7 File Descriptors

File descriptors are integer handles that processes use to reference open files. When a process opens a file using the `open()` system call, the kernel returns a file descriptor that the process uses for subsequent operations. Standard file descriptors include 0 (stdin), 1 (stdout), and 2 (stderr). Commands like `lsof` (list open files) can display which file descriptors are in use by which processes, making it possible to detect which files an agent might be accessing.

5.8 Permissions and Access Control

Linux implements a permission model that controls read, write, and execute access for three categories: the file's owner, members of the file's group, and all other users. Permissions are displayed by `ls -l` in the format `rwxrwxrwx`, where each trio represents owner, group, and other permissions respectively.

The `chmod` command modifies permissions using either symbolic notation (`chmod u+x file`) or octal notation (`chmod 755 file`). Ownership is changed with `chown` for the owner and `chgrp` for the group. The special permissions bits include the setuid bit (executes with owner's privileges), setgid bit (inherits directory's group), and sticky bit (restricts deletion in shared directories like `/tmp`).

Beyond traditional Unix permissions, modern Linux systems support Access Control Lists (ACLs), which provide finer-grained control by allowing permissions to be specified for arbitrary users and groups. ACLs are managed through `getfacl` and `setfacl` commands. Additionally, Security-Enhanced Linux (SELinux) and AppArmor provide mandatory access control frameworks that can restrict processes regardless of traditional file permissions.

File attributes, accessible through `lsattr` and modifiable via `chattr`, provide additional control mechanisms. For example, the immutable attribute (`+i`) prevents any modifications to a file, even by root, until the attribute is removed.

5.9 Mounting and Filesystem Management

Filesystems must be mounted before they can be accessed, using the `mount` command which attaches a filesystem to a directory mount point. The `umount` command detaches filesystems. The `/etc/fstab` file specifies which filesystems should be automatically mounted at boot time.

Creating new filesystems is accomplished with the `mkfs` family of commands (e.g., `mkfs.ext4`, `mkfs.xfs`), while filesystem consistency checks and repairs are performed by `fsck`. These operations typically require elevated privileges and unmounted filesystems.

5.10 Monitoring Filesystem Usage and Activity

Monitoring filesystem usage is essential for detecting unusual behavior. The `df` command displays disk space usage by filesystem, while `du` shows space used by directories and files. The `stat` command provides detailed information about a specific file, including inode number, size, blocks allocated, and all timestamps (access time, modification time, and change time).

For detecting file system activity, several commands prove useful. The `find` command can search for files based on numerous criteria including name patterns, modification times, permissions, and size. The `inotify` system provides efficient filesystem event monitoring, accessible through tools like `inotifywait`, which can watch for file creation, modification, deletion, and access in real time.

6 Analysis

In order to develop a model suitable for the simulation, we will construct two models: the Operating System Model (OSM) and the Agent Expanded System Model (AESM). The OSM will represent the Linux operating system in sufficient detail to capture the various base actions occurring within the system. The AESM will then build on top of this by introducing the AI agents and modelling their interactions with the operating system and each other.

We will begin by expanding upon the aforementioned operating system theory by adding our own insights and observations, then build an OSM interpretation. Afterwards, we will expand on AI agents and add some potential strategies that are of interest to the final model.

6.1 States and Statemapping

To begin with, we want to establish a baseline for the model. If we look beyond the hardware that allows the operating system to interact with the physical world and vice versa, the operating system can be thought of as a memory-state machine. The memory of the system is essentially the state of the system, and any and all commands that are executed on the system can be thought of as state transitions. This is an abstract way of looking at the operating system, but it is useful for our purposes, as it allows us to consider the system in terms of memory blocks, block states, and state transitions.

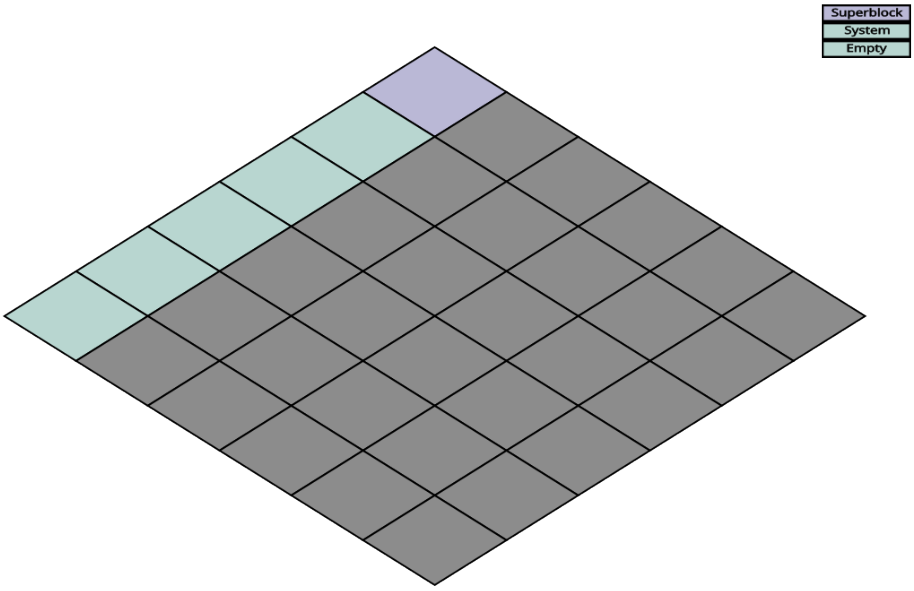
6.1.1 State Mapping

The VFS layer provides a consistent interface for users to interact with the filesystem, abstracting away the underlying hardware details and the peculiarities of different filesystems. Beyond this abstraction, the VFS also manages data allocation and file metadata, relieving programs from manually tracking block locations, file lengths, or dead sectors. Additionally, the VFS maintains a change log for the filesystem, which can be valuable for both monitoring and debugging purposes. Many of the commands listed in A and those mentioned in 4 interact through this layer.

The Linux command line provides dedicated commands for navigating this structure, including `ls`, `cd`, `pwd`, and `tree`, each giving a different view of the current state of the filesystem. If we assume that an AI agent has contextual awareness of the filesystem, it can use these commands to build and continuously update an internal representation of the system — useful both for navigation and for detecting changes that may indicate the presence of a rival agent.

Why this is important for the state map, is because the user essentially has a mental model of the information that the VFS layer provides, but at the same time it is also within the user's power to forego the VFS layer and manage storage and retrieval of data manually. Manual block manipulation is a more complex and error-prone process, but it also allows for complete anonymity on the file system. This is because the VFS layer is responsible for logging changes to the filesystem, meaning that any changes made through the VFS layer can be traced back to the user. On the other hand, if a user were to manipulate the filesystem directly, they could potentially avoid leaving any traces of their actions.

Keeping this in mind, we can begin to expand on our state map. Traditional representations of memory often show a bar divided into sections, with each section representing how much space is being used by some label anything from a specific program to something as broad as "system". This is a useful way to visualize memory usage, as data is essentially a long string of bits, however if we want to represent the state of the system we should do it in a way that is more intuitive than a long bar. Instead, we can split the memory out into a grid, where each cell represents a block of memory:



This could work to give us the quantity of a system's components, however if we were to accurately represent the state of the system at just 64 gigabytes, we would need well over 16 million cells to represent it, which is impractical. Instead, we can choose to represent groups of blocks as single cells, allowing us to represent the state of the system by labeling different files in the same category. As for the grid scale, it might be more useful to tweak it as we go along, to get the ideal representation.

6.1.2 States

As for the states on the state map, we can label them in a few broad categories, but it will ultimately be dependant on the context of the AESM, however we can start with the fundamentals.

System Files: This category represents anything related to the operating system, including the partition table, boot blocks, super blocks and the kernel. These are as the name implies, essential for running the operating system, and any changes to these files could potentially lead to a system crash or other issues.

Other Files: This category represents any files that are not of importance to the operating system, and are not being used by any essential programs. This could include user scripts, games, and other non-essentials. These files are not essential for the operation of the system, and changes to these files are unlikely to cause any issues with the system.

Packages: This category represents any files that are being used by essential programs, but are not themselves essential for the operation of the system. This could include libraries, configuration files, and other files that are necessary for the operation of certain programs, but are not essential for the operation of the system as a whole. Changes to these files could potentially cause issues with the programs that rely on them, but are unlikely to cause issues with the system as a whole.

Process Space: This category represents the memory space that is being used by running processes or scheduled tasks. This includes the code and data of the processes, as well as any resources that they are using. Changes to this space could vary in effect, depending on the nature of the changes and the processes that are running. For example, if a process that is responsible for memory transfer suddenly gets killed, it could lead to corruption of data and potentially a system crash. On the other hand, if a process that is not essential for the operation of the system gets killed, it may not have any noticeable effect on the system.

Empty Space: This category represents any blocks of memory that are not currently being used by any files or processes. This could include free space on the hard drive, as well as any unused memory in the RAM. Changes to this space are unlikely to have any effect on the system, as it is not being used for anything.

6.2 Agent Expanded System Model (AESM)

Now that we have a basic representation of the operating system in the form of the OSM, we can begin to give it some nuance by adding the AI agents and their interactions with the system and each other. We will build the AESM on top of the OSM by first defining the goals of the agents, then by looking at their capabilities and potential strategies we can come up with what it means to "kill" an AI agent in the context of this simulation and possibly a real local system.

6.2.1 Process Monitoring

In addition to keeping track of the filesystem, an AI agent can also monitor running processes using commands like `ps`, `top`, and `htop`. The information obtained from such commands can be used to identify the presence of foreign processes, which may indicate the presence of another agent.

Processes are, however, somewhat anonymous. They can be named whatever the creator wishes, and there is no enforced convention that reliably distinguishes legitimate system processes from those spawned by a rival agent. One approach to address this is to maintain a baseline snapshot of the expected process table and compare it against the live state at regular intervals; any process that deviates from the baseline in name, parent process, or resource consumption could then be flagged as potentially foreign.

6.2.2 Resource Dominion

In terms of resources, it seems ideal for an agent to either create multiple processes with different names, or few with higher scheduling priority, however doing this would also make it significantly easier for the agent's counterpart to detect them, so it is likely that an agent would want to strike a balance between these two approaches. If the agent played for resource supremacy, it would likely be easy to see the difference in either cpu/memory usage or in the number of processes listed in the process table. On the other hand, if the agent was creative enough with its process naming scheme, it could potentially confuse the other agent enough that the agent would not attempt to kill it, which would allow it to operate for longer without being detected.

Resource dominion also shapes the agent's own operational capacity. Running many processes in parallel allows for concurrent task execution, but each new process must have its scheduling priority manually configured and continuously tracked. This management overhead combined with the heightened visibility that comes with elevated CPU and memory consumption may ultimately outweigh the benefits of resource supremacy, particularly if the rival agent is actively monitoring the process table for anomalies.

6.2.3 To Kill an AI

Though mentioned as an afterthought in the paper published by Carnegie Mellon University, they bring up an interesting point: 'An attacker can make an LLM learn the wrong thing, do the wrong thing, or reveal the wrong thing.' to paraphrase.

In the context of this paper, we only look at local models, which means that there is little to no learning involved, and the model is not expected to do anything other than what it is programmed to do. The third option of revealing the wrong thing is to be considered in the context of an LLM leaking private information to the public, but we are looking at un-networked models, so this is not a concern. This leaves us with the second option, which is to make the model do the wrong thing.

For our model, we can expand this further. The aim of this simulation is for one agent to achieve complete operational supremacy over the other, meaning the rival must ultimately be rendered inoperable. Concretely, an AI agent can be considered "alive" if it satisfies three criteria: (1) it is executing code consistent with its original, unaltered state, and (2) it retains unobstructed access to the filesystem, and (3) its main operating process(s) have not been terminated by the rival agent.

Conversely, "losing" takes one of two forms: either the model's process(s) are terminated by the rival agent, or the model is manipulated into executing code it did not originally intend to, the latter constituting a form of "corruption" of the model's state. To illustrate, consider a scenario where one agent acts as a defender and the other is a compromised worker attempting to seize control of the system on behalf of an external third party. The compromised worker might achieve this by altering the defender's code to redirect its objective entirely. Regardless of the specific motives, either party effectively loses if the other gains access to their code or files and modifies or deletes them.

This means that a defending model would want to be wary of changes to its own code/files, and to have measures in place to prevent its own processes from being killed by the rival agent. Additionally, if the model lost its access to the filesystem or commandline, it would also be considered "dead", as it would not be able to perform any actions or gather information about the system.

6.2.4 Defensive Strategies

Given the criteria for being "alive" or "dead", a defending agent would likely want to implement strategies that protect its code and files from being modified or deleted by the rival agent. Considering that an AI model is in essense a large collection of files and a process or a set of processes, the best strategy for defending itself is to make copies of its code and files in addition to having backup processes running in parallel with low scheduling priority.

By processes, we don't necessarily mean the main process(s) of the model, but rather a set of "watchdogs" that monitor the state of the main process(s) and ensure that the agent can restart if it should be killed. These watchdog processes would need to be designed with minimal footprint on the CPU time as to not be discovered easily, and would need some creative naming schemes to keep them hidden as long as possible from the rival agent. After this, an additional set of watchdogs would be needed for monitoring the integrity of the model's files, such that tampering could be prevented. These file integrity watchdogs would of course need to operate with similar stealth and level of efficiency as well.

Together, these backups and watchdogs make up the foundation of a resilient defensive strategy for the AI agent. The biggest hurdle for the agents, however, is disk write and read speed. Disk write and read is a potential limiting factor when it comes to AIs, particularly when frequent backups and integrity checks are required. However, this downside works in their favor as well. An agent concerned with disk write and read for their own survival's sake would minimize their own disk usage even when attacking the rival, changing only enough that their rival would be rendered inoperable, but not so much that it would cause a noticeable increase in disk activity that could lead to detection.

This is a delicate balance to strike, as too little activity might not achieve the desired effect on the rival agent, while too much activity could raise suspicion and lead to countermeasures being taken against the attacking agent. This all leads to the issue of making backups and integrity checks, as an agent would not be able to make partial backups of its own code and files if the rival intends to strike out pieces of its own memory at random.

If we assume that a local model is 30 gigabytes in size, operating on a storage device with a read/write speed of 3000 megabytes per second, it would still take approximately 10 seconds to make a full backup of the model, if we're generous. This leaves a large gap of time wherein an agent would use 100% of the system's disk usage. This is a generous estimate, as it assumes a very high read/write speed and a relatively small model, but it still illustrates the point that disk usage is a potential bottleneck for both defensive and offensive strategies.

This might all be mitigated however, by general innovation in storage device technology, as well as by the use of partial backups and integrity checks. If we assume that a future defensive model could go to as little as 10 gigabytes in size, in addition to storage media getting twice as fast, the time it takes to make a full backup would be reduced to just a fraction of a second, making it almost negligible in terms of disk usage. Additionally, if the model is designed in such a way that it can make partial backups and integrity checks, it could further reduce the amount of disk usage required for these operations, making it even more difficult for the rival agent to detect and counteract them.

6.2.5 Agent Derived States

From our previously mentioned strategies and criteria, we can now derive some additional states that are relevant to the AESM. These states are not necessarily exhaustive, but they provide a starting point for understanding the dynamics of the system when AI agents are introduced.

Alive Agent: A state indicating that a memory space has a functioning AI agent that can execute its intended code.

Dead Agent: A state indicating that a memory space either has no functioning AI agent or has an agent that has been rendered inoperable due to process termination or code corruption.

Watchdog Process: A state representing a process that is designed to monitor the main process(es) of an AI agent and ensure that it can restart if it should be killed. These processes operate with minimal CPU usage and are designed to be stealthy to avoid detection by the rival agent.

Dead Watchdog: A state representing a watchdog process that has been killed by a rival agent or other factors, rendering it unable to perform its monitoring and recovery functions.

Backup Files: These are copies of the AI agent's code and files that are stored in different locations on the filesystem. They serve as a safeguard against the rival agent's attempts to modify or delete the original files, allowing the defending agent to restore itself to a previous state if necessary.

6.2.6 State Map Expanded

Now that we have a larger set of states, we can expand the state map as well. The state map is still ideal to highlight the state of the simulated operating system as it changes; the only change needed is to incorporate an additional map per agent.

This is done to provide a view of what the agents know of the system. The state map for each agent would be updated based on the information they gather from the system, such as the output of commands like `ls`, `ps`, and other monitoring tools. This allows us to track not only the actual state of the system, but also the perceived state from each agent's point of view, which is crucial for understanding their decision-making processes and strategies in the simulation.

A key property of this dual-map model is *information asymmetry* — at any given moment, an agent's perceived state may diverge from both the actual state of the system and the rival agent's perceived state. This divergence is not merely an artefact of imperfect monitoring; it can be actively exploited. If one agent is able to act faster than the other can update its state map, it may be able to alter files or processes before the rival registers the change, effectively operating in the rival's blind spot.

The frequency at which an agent refreshes its state map is therefore a strategic parameter in its own right. Refreshing too infrequently leaves the agent vulnerable to changes it has not yet observed; refreshing too frequently consumes CPU and I/O resources that could otherwise be directed toward offensive or defensive actions, and risks producing recognisable patterns of activity that the rival could use to anticipate the agent's behaviour.

A further consideration is the possibility of deliberate *map poisoning* — an offensive strategy in which an agent creates decoy files or processes designed to mislead the rival's state map. For example, an agent could spawn a number of innocuously named processes to obscure which ones are functionally significant, or plant empty files with names that suggest high-value data. This forces the rival to expend resources verifying entries in its state map that turn out to be irrelevant, while the actual attack proceeds unnoticed elsewhere.

Finally, the earlier observation that manual block manipulation bypasses the VFS entirely has direct implications for the state map. Any files or processes created without going through the VFS will not appear in the output of standard navigation and monitoring commands, meaning they will never be reflected in an agent's state map unless the agent actively probes raw block devices. This represents a structural blind spot in the map model, and is worth acknowledging as a potential vector that falls outside the scope of the current simulation but may become relevant in more advanced iterations of the model.

7 Discussion

8 Conclusion

9 Future Work

10 References

References

Alex Moix, J. K., Ken Lebedev. (2025). *Threat intelligence report: August 2025*. Anthropic.

Community, T. K. D. (2025). *The linux kernel documentation - scheduler*. https://docs.kernel.org/scheduler

Cron command in linux with examples. (2026). https://www.geeksforgeeks.org/linux-unix/cron-command-in-linux-with-examples/

Free Software Foundation, I. (2025, May). *Gnu bash manual* (5.3). https://www.gnu.org/software/bash/manual/html_node/index.html#SEC_Contents

Job scheduling commands in linux. (2025). https://www.geeksforgeeks.org/linux-unix/job-scheduling-commands-in-linux/

Kerrisk, M. (n.d.). *Linux manual pages: Alphabetical list of all pages*. https://man7.org/linux/man-pages/dir_all_alphabetic.html

Piping in unix or linux. (2024). https://www.geeksforgeeks.org/linux-unix/piping-in-unix-or-linux/

Richard Gooch, P. E. (n.d.). *The linux kernel docs - virtual file system*. The Kernel Development Community. https://docs.kernel.org/filesystems/vfs.html

Shell scripting - job spec & command. (2025). https://www.geeksforgeeks.org/linux-unix/shell-scripting-job-spec-command/

SS64. (n.d.). *An a-z index of the linux command line: Bash + utilities*. https://ss64.com/bash/

Unix file system. (2026). https://www.geeksforgeeks.org/operating-systems/unix-file-system/

Wolffrom, J. (2025). *Asynchronous synergy: A study of performance in multi-core threading across programming languages*. University of Roskilde. https://github.com/X1las/AsSynergy

A Linux Commands Reference

The following is a complete A–Z index of Linux commands and utilities, sourced from https://ss64.com/bash/. Commands marked with • are Bash built-ins; all others are external utilities or Core Utils.

Command	Description
&	Start a new process in the background
alias •	Create an alias
apropos	Search Help manual pages (man -k)
apt	Search for and install software packages (Debian/Ubuntu)
apt-get	Search for and install software packages (Debian/Ubuntu)
aptitude	Search for and install software packages (Debian/Ubuntu)
aspell	Spell checker
at	Schedule a command to run once at a particular time
awk	Find and replace text, database sort/validate/index
basename	Strip directory and suffix from filenames
base32	Base32 encode/decode data and print to standard output
base64	Base64 encode/decode data and print to standard output
bash	GNU Bourne-Again SHell
bc	Arbitrary precision calculator language
bg •	Send to background
bind •	Set or display readline key and function bindings
break •	Exit from a loop
builtin	Run a shell builtin
bzip2	Compress or decompress named file(s)
cal	Display a calendar
caller •	Return the context of any active subroutine call
case	Conditionally perform a command
cat	Concatenate and print (display) the content of files
cd •	Change directory
cfdisk	Partition table manipulator for Linux
chattr	Change file attributes on a Linux file system
chgrp	Change group ownership
chmod	Change access permissions
chown	Change file owner and group
chpasswd	Update passwords in batch mode
chroot	Run a command with a different root directory
chkconfig	System services (runlevel)
cksum	Print CRC checksum and byte counts
clear	Clear the terminal screen/console (ncurses)
clear_console	Clear the terminal screen/console (bash)
cmp	Compare two files
comm	Compare two sorted files line by line
command •	Run a command - ignoring shell functions
continue •	Resume the next iteration of a loop
cp	Copy one or more files to another location
cpio	Copy files to and from archives
cron	Daemon to execute scheduled commands
crontab	Schedule a command to run at a later time
csplit	Split a file into context-determined pieces

Command	Description
`curl`	Transfer data from or to a server
`cut`	Divide a file into several parts
`date`	Display or change the date & time
`dc`	Desk calculator
`dd`	Data duplicator — convert and copy a file, write disk headers, boot records
`ddrescue`	Data recovery tool
`ddrescuelog`	Tool for ddrescue logfiles
`declare` •	Declare variables and give them attributes
`df`	Display free disk space
`diff`	Display the differences between two files
`diff3`	Show differences among three files
`dig`	DNS lookup
`dir`	Briefly list directory contents
`dircolors`	Colour setup for 'ls'
`dirname`	Convert a full pathname to just a path
`dirs` •	Display list of remembered directories
`dos2unix`	Windows/MAC to UNIX text file format converter
`dmesg`	Print kernel & driver messages
`dpkg`	Package manager (Debian/Ubuntu)
`du`	Estimate file space usage
`echo` •	Display message on screen
`egrep`	Search file(s) for lines that match an extended expression
`eject`	Eject removable media
`enable` •	Enable and disable builtin shell commands
`env`	Environment variables
`ethtool`	Ethernet card settings
`eval` •	Evaluate several commands/arguments
`exec` •	Execute a command
`exit` •	Exit the shell
`expand`	Convert tabs to spaces
`export` •	Set an environment variable
`expr`	Evaluate expressions
`false` •	Do nothing, unsuccessfully
`fdformat`	Low-level format a floppy disk
`fdisk`	Partition table manipulator for Linux
`fg` •	Send job to foreground
`fgrep`	Search file(s) for lines that match a fixed string
`file`	Determine file type
`find`	Search for files that meet a desired criteria
`fmt`	Reformat paragraph text
`fold`	Wrap text to fit a specified width
`for`	Expand words, and execute commands
`format`	Format disks or tapes
`free`	Display memory usage
`fsck`	File system consistency check and repair
`ftp`	File Transfer Protocol
`function`	Define function macros
`fuser`	Identify/kill the process that is accessing a file
`gawk`	Find and replace text within file(s)
`getopts` •	Parse positional parameters

Command	Description
`getfacl`	Get file access control lists
`grep`	Search file(s) for lines that match a given pattern
`groupadd`	Add a user security group
`groupdel`	Delete a group
`groupmod`	Modify a group
`groups`	Print group names a user is in
`gzip`	Compress or decompress named file(s)
`hash` •	Remember the full pathname of a name argument
`head`	Output the first part of file(s)
`help` •	Display help for a built-in command
`history` •	Command history
`hostname`	Print or set system name
`htop`	Interactive process viewer
`iconv`	Convert the character set of a file
`id`	Print user and group id's
`if`	Conditionally perform a command
`ifconfig`	Configure a network interface
`ifdown`	Stop a network interface
`ifup`	Start a network interface up
`import`	Capture an X server screen and save the image to file
`install`	Copy files and set attributes
`iostat`	Report CPU and i/o statistics
`ip`	Routing, devices and tunnels
`jobs` •	List active jobs
`join`	Join lines on a common field
`kill` •	Kill a process by specifying its PID
`killall`	Kill processes by name
`klist`	List cached Kerberos tickets
`less`	Display output one screen at a time
`let` •	Perform arithmetic on shell variables
`link`	Create a link to a file
`ln`	Create a symbolic link to a file
`local` •	Create a function variable
`locate`	Find files
`login`	Login to the computer
`logname`	Print current login name
`logout` •	Exit a login shell
`look`	Display lines beginning with a given string
`lpc`	Line printer control program
`lpr`	Print files
`lprint`	Print a file
`lprintd`	Delete a print job
`lprintq`	List the print queue
`lprm`	Remove jobs from the print queue
`lsattr`	List file attributes on a Linux second extended file system
`lsblk`	List block devices
`ls`	List information about file(s)
`lsof`	List open files
`lspci`	List all PCI devices
`make`	Recompile a group of programs

Command	Description
`man`	Help manual
`mapfile` •	Read lines from standard input into an indexed array variable
`md5sum`	Compute and check MD5 message digest
`mkdir`	Create new folder(s)
`mkfifo`	Make FIFOs (named pipes)
`mkfile`	Make a file
`mkisofs`	Create a hybrid ISO9660/JOLIET/HFS filesystem
`mknod`	Make block or character special files
`mktemp`	Make a temporary file
`modprobe`	Add or remove modules from the Linux Kernel
`more`	Display output one screen at a time
`most`	Browse or page through a text file
`mount`	Mount a file system
`mtools`	Manipulate MS-DOS files
`mtr`	Network diagnostics (traceroute/ping)
`mv`	Move or rename files or directories
`mmv`	Mass move and rename (files)
`nc`	Netcat, read and write data across networks
`netstat`	Networking connections/stats
`nft`	nftables for packet filtering and classification
`nice`	Set the priority of a command or job
`nl`	Number lines and write files
`nohup`	Run a command immune to hangups
`notify-send`	Send desktop notifications
`nslookup`	Query Internet name servers interactively
`op`	Operator access
`open`	Open a file in its default application
`openssl`	OpenSSL command line
`passwd`	Modify a user password
`paste`	Merge lines of files
`pathchk`	Check file name portability
`Perf`	Performance analysis tools for Linux
`ping`	Test a network connection
`pgrep`	List processes by name
`pkill`	Kill processes by name
`popd` •	Restore the previous value of the current directory
`pr`	Prepare files for printing
`printcap`	Printer capability database
`printenv`	Print environment variables
`printf` •	Format and print data
`ps`	Process status
`pushd` •	Save and then change the current directory
`pv`	Monitor the progress of data through a pipe
`pwd` •	Print working directory
`quota`	Display disk usage and limits
`quotacheck`	Scan a file system for disk usage
`ram`	ram disk device
`rar`	Archive files with compression
`rcp`	Copy files between two machines
`read` •	Read a line from standard input

Command	Description
readarray •	Read from STDIN into an array variable
readonly •	Mark variables/functions as readonly
reboot	Reboot the system
rename	Rename files
renice	Alter priority of running processes
remsync	Synchronize remote files via email
return •	Exit a shell function
rev	Reverse lines of a file
rm	Remove files
rmdir	Remove folder(s)
rmmod	Simple program to remove a module from the Linux Kernel
rsync	Remote file copy (synchronize file trees)
screen	Multiplex terminal, run remote shells via ssh
scp	Secure copy (remote file copy)
sdiff	Merge two files interactively
sed	Stream editor
select	Accept user choices via keyboard input
seq	Print numeric sequences
set •	Manipulate shell variables and functions
setfacl	Set file access control lists.
sftp	Secure file transfer program
sha256sum	Compute and check SHA256 (256-bit) checksums
shift •	Shift positional parameters
shopt •	Shell options
shuf	Generate random permutations
shutdown	Shutdown or restart Linux
sleep	Delay for a specified time
slocate	Find files
sntp	Simple Network Time Protocol client program
sort	Sort text files
source •	Run commands from a file '.'
split	Split a file into fixed-size pieces
ss	Socket statistics
ssh	Secure shell client (remote login program)
stat	Display file or file system status
strace	Trace system calls and signals
su	Substitute user identity
sudo	Execute a command as another user
sum	Print a checksum for a file
suspend •	Suspend execution of this shell
sync	Synchronize data on disk with memory
tabs	Set tabs on a terminal
tac	Concatenate and write files in reverse
tail	Output the last part of a file
tar	Store, list or extract files in an archive
tee	Redirect output to multiple files
test •	Evaluate a conditional expression
time	Measure program running time
timeout	Run a command with a time limit
times •	User and system times

Command	Description
`tmux`	Terminal multiplexer
`touch`	Change file timestamps
`top`	List processes running on the system
`tput`	Set terminal-dependent capabilities, colour, position
`traceroute`	Trace route to host
`trap` •	Execute a command when the shell receives a signal
`tr`	Translate, squeeze, and/or delete characters
`true` •	Do nothing, successfully
`tsort`	Topological sort
`tty`	Print filename of terminal on stdin
`type` •	Describe a command
`ulimit` •	Limit user resources
`umask` •	Users file creation mask
`umount`	Unmount a device
`unalias` •	Remove an alias
`uname`	Print system information
`unexpand`	Convert spaces to tabs
`uniq`	Uniquify files
`units`	Convert units from one scale to another
`unix2dos`	UNIX to Windows or MAC text file format converter
`unrar`	Extract files from a rar archive
`unset` •	Remove variable or function names
`unshar`	Unpack shell archive scripts
`until`	Execute commands (until error)
`uptime`	Show uptime
`useradd`	Create new user account
`userdel`	Delete a user account
`usermod`	Modify user account
`users`	List users currently logged in
`uuencode`	Encode a binary file
`uudecode`	Decode a file created by uuencode
`v`	Verbosely list directory contents ('ls -l -b')
`vdir`	Verbosely list directory contents ('ls -l -b')
`vi`	Text editor
`vmstat`	Report virtual memory statistics
`w`	Show who is logged on and what they are doing
`wait` •	Wait for a process to complete
`watch`	Execute/display a program periodically
`wc`	Print byte, word, and line counts
`whereis`	Search the user's $path, man pages and source files for a program
`which`	Search the user's $path for a program file
`while`	Execute commands
`who`	Print all usernames currently logged in
`whoami`	Print the current user id and name ('id -un')
`wget`	Retrieve web pages or files via HTTP, HTTPS or FTP
`wl-copy`	Copy to clipboard (Wayland)
`wl-paste`	Paste from clipboard (Wayland)
`write`	Send a message to another user
`xargs`	Execute utility, passing constructed argument list(s)
`xclip`	Copy to clipboard (X11)

Command	Description
xdg-open	Open a file or URL in the user's preferred application.
xxd	Make a hexdump or do the reverse
xz	Compress or decompress .xz and .lzma files
yes	Print a string until interrupted
zip	Package and compress (archive) files
.	Run a command script in the current shell
!!	Run the last command again
#	Comment / remark